Задача 1. Хранение строк

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать систему хранения строк в динамической памяти.
Система должна обрабатывать три вида запросов/операций:

- **0** - создать строку: указывается длина создаваемой строки и собственно содержимое строки. Нужно выделить блок памяти при помощи ***malloc*** и записать в него содержимое.
- **1** - удалить строку: указывается идентификатор ранее созданной строки. Нужно освободить память, выделенную ранее для этой строки, при помощи free.
- **2** - вывести строку: указывается идентификатор ранее созданной строки. Нужно распечатать содержимое этой строки в выходной файл.
- **3** - Сколько символов в строке: указывается идентификатор ранее созданной строкии один символ. Нужно вывести в выходной файл одно целое число: сколько раз этот символ встречается в указанной строке.

Идентификатором строки является номер запроса на её создание в общей нумерации запросов. При обращении к строке по идентификатору гарантируется, что эта строка ещё **не** была удалена.
Замечание: не все созданные строки удаляются в запросах.
В целях аккуратности удалите все остающиеся строки при помощи ***free*** в конце программы.

Формат входного файла

В первой строке записано целое число **N** - количество запросов ($1 \le N \le 10^5$).
В остальных **N** строках описаны запросы, по одному запросу в строке.
Запрос начинается с целого числа t, обозначающего тип запроса.
Если **t = 0**, то это запрос создания, и тогда далее указана длина строки l и сама строка ($1 \le l \le 10^5$).
Если **t = 1**, то это запрос удаления, а если **t = 2** - то это запрос на вывод.
В обоих случаях далее указано целое число k - идентификатор строки ($0 \le k < N$).
Если **t = 3**, то это запрос о количестве символов, и тогда далее указан идентификатор строки k и ещё один символ через пробел.
Все строки состоят из произвольных печатаемых символов ASCII кроме пробела (коды от 33 до 126 включительно). То же верно и про символы в запросах на количество.
Сумма всех длин l по всем запросам создания не превышает $5 \cdot 10^5$.
Суммарный размер всех строк, которые вам нужно вывести, не превышает $5 \cdot 10^5$.
Сумма длин строк по всем запросам о количестве символов не превышает 10^8.

Пример

input.txt	output.txt
12 0 3 aba 2 0 3 0 a 3 0 b 1 0 0 6 malloc 0 5 %d%s% 2 1 2 2 1 1 3 2 % 2 2	aba 2 1 malloc %d%s% 3 %d%s%

Пояснение к примеру

Сначала создаётся строка aba с идентификатором 0.
Далее она распечатывается, и выводится количество букв a и b в ней.
Наконец, нулевая строка удаляется.
Затем создаются ещё две строки: строка malloc и строка %d%s% с идентификаторами 1 и 2 соответственно.
Потом они распечатываются, и строка malloc удаляется.
В конце выводится количество процентов в строке %d%s% и она распечатывается ещё раз.

Задача 2. Бинарный поиск

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется реализовать бинарный поиск элемента с заданным значением в отсортированном массиве.

В первой строке записано одно целое число N — размер отсортированного массива ($1 \le N \le 2 \cdot 10^5$). Далее записаны элементы массива A_i (N целых чисел, $|A_i| \le 10^9$). Затем записано целое число Q — количество запросов, которые нужно обработать ($1 \le N \le 2 \cdot 10^5$). В остальных Q строках записаны целые числа X_j, определяющие запросы на поиск.

Каждый запрос нужно обрабатывать следующим образом. Сначала нужно прибавить к записанному в файле числу X_j ответ на предыдущий запрос R_{j-1}, получив $Y_j = X_j + R_{j-1}$. Затем нужно найти в массиве A элемент, равный Y_j: его индекс будет ответом R_j для этого запроса. Если таких элементов много, то в качестве ответа R_j следует выбрать самый большой индекс. Если таких элементов нет, то ответ R_j равен -1.

Элементы массива нумеруются индексами от 0 до $N - 1$. Для первого запроса предыдущего ответа нет, так что полагаем $Y_0 = X_0$.

Пример

input.txt	output.txt
10 1 1 3 4 4 7 8 10 10 12 10 1 2 3 4 5 6 7 8 9 10	1 2 -1 2 5 -1 -1 5 -1 -1

Пояснение к примеру

Первый запрос: нужно найти значение $X_0 = Y_0 = 1$. Таких элементов два, и они имеют индексы 0 и 1. В данной задаче нужно всегда выбирать максимальных индекс, если выбор есть, поэтому ответ A_0 равен 1.

Для следующего запроса задано число $X_1 = 2$. Прибавляем к нему предыдущий ответ $A_0 = 1$, и получаем число $Y_1 = 3$, которое нужно искать. Такой элемент есть в массиве под индексом 2, так что выводим ответ $A_1 = 2$.

Для следующего запроса указано $X_2 = 3$. Прибавляем предыдущий ответ $A_1 = 2$, и получаем, что нужно искать $Y_2 = 5$. Такого числа нет, так что выводим ответ $A_2 = -1$.

Теперь рассмотрим запрос $X_3 = 4$. Сперва прибавляем предыдущий ответ $A_2 = -1$, получаем число $Y_3 = 3$, которое нужно искать. Значит ответ $A_3 = 2$. И так далее....

Задача 3. Язык «Студенческий С»

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Студенты очень любят вставать и ходить на пары в 9 утра (нет). В мыслях они очень амбициозны и хотят разработать язык программирования чтобы быстрее писать на Си и дольше спать. Они подумали и придумали:

- Все нужные директивы #include заменяются на одно слово puk (как в примере).
- Ключевое слово для автоматического определения типа переменных (__auto_type) заменяется на hiya (как в примере).
- Функции freopen заменяется на yaz (как в примере).
- Ключевое слово return заменяется на r (как в примере).

Правда студенты понимали, что они часто невнимательны и могут ошибаться так как отвлекаются на CS и DOTA 2. Они подумали, что hiya, puk и yaz они могут написать с одной ошибкой. Гарантируется, что они не смешиваются

Ваша задача — написать программу, которая преобразует код на «Студенческом С» в стандартный С, используя конечный автомат для обработки лексем.

Пример

input.txt	output.txt

puk
 	
int main(void) {
  yaz
 	
  hiya a = 56;
  hiya b = 39;
 	
  printf("%d", a + b);
  r 0;
}
	#include <stdio.h>
	#include <stdlib.h>
	#include <memory.h>
	#include <ctype.h>
	#include <math.h>
	#include <string.h>
	
	int main(void) {
	  freopen("input.txt", "r", stdin);
	  freopen("output.txt", "w", stdout);
	
	  __auto_type a = 56;
	  __auto_type b = 39;
	
	  printf("%d", a + b);
	  r 0;
	}

Задача 4. Болезнь «Хокинга»

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Стивен Хокинг (1942–2018) — выдающийся физик-теоретик, космолог и автор, известный своими работами о черных дырах и книге "Краткая история времени". В 21 год у него диагностировали боковой амиотрофический склероз (БАС), нейродегенеративное заболевание, поражающее двигательные нейроны, что привело к постепенному параличу. Хокинг прожил с БАС более 55 лет, что крайне атипично, так как обычно заболевание прогрессирует за 2–5 лет. Одной из причин семейного БАС является мутация в гене C9orf72 — экспансия повторов GGGGCC (>30 считается патологической), вызывающая токсичность РНК и белков.
Требуется реализовать программу на языке C, которая использует алгоритм Кнута-Морриса-Пратта (КМП) для поиска гексануклеотидных повторов GGGGCC в последовательностях ДНК/РНК. Список поддерживает операции добавления и удаления узлов, а каждая последовательность проверяется на патологию (>30 повторов — патологическая).

Формат входного файла

Первая строка: целое число T ($0 \le T \le 10^9$) — количество нуклеотидов в последовательности РНК.

Далее идет строка состоящая из нуклеотидов:

- A — Аденин
- C — Цитозин
- G — Гуанин
- U — Урацил (в ДНК вместо урацила используется тимин — T)

Формат выходного файла

Вывести:

1. Pathogenic – если есть патология (>30 повторов GGGGCC)
2. Non-pathogenic – если ее нет

Пример

input.txt	output.txt
20 GGGGCCAGGGGCCUUGGGGC	Non-pathogenic
792 GGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGG GCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCU UGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGG GCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCC AGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGG GGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGC AUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUG GGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGC CUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAG GGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGG CCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAU GGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGG GCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCU UGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGG GCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCC AGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAUGG GGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUGGGGC AUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGCCUUG GGGCAUGGGGCCAGGGGCCUUGGGGCAUGGGGCCAGGGGC CUUGGGGCAUGGGGCCAGGGGCCUUGGGGCAU	Pathogenic

Задача 5. Поиск ближайшего

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется реализовать поиск в отсортированном массиве элемента, ближайшего к заданному значению.

В первой строке записано одно целое число N — размер отсортированного массива ($1 \le N \le 2 \cdot 10^5$). Далее записаны элементы массива A_i (N целых чисел, $|A_i| \le 10^9$). Затем записано целое число Q — количество запросов, которые нужно обработать ($1 \le N \le 2 \cdot 10^5$). В остальных Q строках записаны целые числа Y_j, определяющие запросы на поиск.

Запросы нужно обрабатывать следующим образом. Для каждого числа Y_j нужно найти в массиве A такой элемент, чтобы разность между ним и Y_j была минимальной. В выходной файл нужно вывести два целых числа через пробел: индекс найденного элемента и полученное минимальное расстояние.

Элементы массива нумеруются индексами от 0 до $N - 1$. Если ближайших элементов несколько, разрешается выводить индекс любого из них.

Важно: В данной задаче ответ для каждого запроса нужно находить независимо от всех остальных запросов за время $O(\log N)$ в худшем случае.

Пример

input.txt	output.txt
10 1 1 3 4 4 7 8 10 10 12 10 1 2 3 4 5 6 7 8 9 10	0 0 2 1 2 0 3 0 4 1 5 1 5 0 6 0 7 1 7 0

Пояснение к примеру

В первом запросе нужно найти значение 1: в массиве два таких элемента, они имеют индексы 0 и 1. Расстояние получается нулевым.

Во втором запросе нужно искать число 2. Ближайшие к нему элементы массива — это число 1 (индексы 0 и 1) и число 3 (индекс 2). Соответственно первое число ответ может быть 0, 1 или 2, а второе (расстояние) должно быть равно единице.

Задача 6. Считалочка

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В кругу стоят N человек, пронумерованных числами от 1 до N в направлении по часовой стрелке. Начиная с первого человека, начинают считать по часовой стрелке, и каждый K-ый, посчитанный выходит из круга. Так делают до тех пор, пока из круга не выйдут все кроме одного человека.
В выходной файл нужно вывести N целых чисел (по одному в строке) — номера людей в порядке их выбывания из круга, включая даже самого последнего.
Важно: в целях обучения, при решении данной задачи требуется хранить стоящих в кругу людей в связном списке любого вида.
Ограничения: $3 \le N \le 5\,000$, $2 \le K < N$.

Пример

input.txt	output.txt
10 3	3 6 9 2 7 1 8 5 10 4

Пояснение к примеру

В кругу 10 человек, начинаем считать с 1-ого, и выбывает каждый третий. На первом круге вылетают 3-ый, 6-ой и 9-ый, остаются люди с номерами 1, 2, 4, 5, 7, 8, 10. На втором круге вылетают 2-ый и 7-ой, остаются номера 1, 4, 5, 8, 10. Дальше вылетают 1-ый и 8-ой, остаются номера 4, 5, 10. Наконец, выбывают 5-ый и 10-ый, а 4-ый остаётся последним.

Задача 7. Длинное умножение

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Во входном файле задано два целых положительных числа, одно в первой строке, второе — во второй строке. Нужно вычислить произведение этих двух чисел и вывести его в выходной файл.

Количество десятичных цифр в каждом входном числе может достигать 1 000. Поэтому требуется реализовать умножение двух длинных чисел “столбиком”.

Пример

input.txt	output.txt
1991 30457	60639887

Задача 8. LGA (Lattice Gas Automata)

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	разумное

Вам требуется смоделировать поведение двумерного решеточного газового автомата (Lattice Gas Automata, LGA) на прямоугольной решетке размером N×M. Каждая ячейка решетки может содержать частицы, движущиеся в одном из четырех направлений: вправо **(1,0,0,0)**, влево (**0,1,0,0)**, вверх **(0,0,1,0)**, вниз **(0,0,0,1)**, или быть пустой **(0,0,0,0)**. Состояние ячейки представлено целым числом в 16-ричном формате, где каждый бит соответствует направлению:

- $8_{16} = 1000_2$ — частица движется вправо,
- $4_{16} = 0100_2$ — частица движется влево,
- $2_{16} = 0010_2$ — частица движется вверх,
- $1_{16} = 0001_2$ — частица движется вниз,
- $0_{16} = 0000_2$ — ячейка пуста.

Числа от

0_{16} до F_{16} представляют комбинации направлений (например, $A_{16} = 1010_2 = (1,0,1,0)$ означает частицы вправо и вверх). На каждом шаге моделирования:

1. Частицы перемещаются в соседние ячейки согласно их направлению.
2. Если в ячейке после перемещения ровно две частицы, они поворачивают на 90 градусов. Например:
    - Пара C = $(8_{16}, 4_{16}) \to (2_{16}, 1_{16})$— (вправо и влево → вверх и вниз)
    - Пара C = $(1_{16}, 4_{16}) \to (8_{16}, 2_{16})$— (вниз и влево → вправо и вверх)
3. Если в ячейке 1, 3 или 4 частицы, они сохраняют свои направления.
4. Пустая ячейка остается

Решетка циклическая: левая граница соединена с правой (по столбцам), верхняя — с нижней (по строкам). Необходимо вычислить состояние решетки после T шагов.

Формат входного файла

В первой строке входного файла содержатся три целых числа N, M и T — количество строк, столбцов решетки и количество шагов моделирования ($1 \le N, M \le 100$, $0 \le T \le 10000$). Далее следуют (N) строк, в каждой из которых задано M целых чисел $a_{i,j}$, где $a_{i,j} \in [0, F]$:

- 8 = (1,0,0,0) — частица движется вправо,
- 4 = (0,1,0,0) — частица движется влево,
- 2 = (0,0,1,0) — частица движется вверх,
- 1 = (0,0,0,1) — частица движется вниз,
- 0 = (0,0,0,0) — ячейка пуста.

Пример: A – две частицы движутся 1 вправо, 1 – вверх.

Формат выходного файла

Выведите N строк, в каждой из которых M чисел в 16-ричном формате без префикса (0x), представляющие состояние решетки после T шагов. Числа в строке разделяются пробелами.

Пример

input.txt	output.txt
3 3 1 8 0 A 0 2 0 1 0 0	6 5 0 0 0 0 0 0 2

Пояснение к примеру

Ячейка (0,0) = (1,0,0,0) — частица движется вправо в (0,1).

Ячейка (0,2) = (1,0,1,0) — две частицы: одна движется вправо в (0,0), другая вверх в (2,2).

Ячейка (1,1) = (0,0,1,0) — частица движется вверх в (0,1).

Ячейка (2,0) = (0,0,0,1) — частица движется вниз в (0,0) (из-за циклической границы).

После перемещения:

(0,0): частицы (2,0) (вниз) и (0,2) (вправо) → 9 = (1,0,0,1).

(0,1): частицы (0,0) (вправо) и (1,1) (вверх) → A = (1,0,1,0).

(2,2): частица (0,2) (вверх) → 2 = (0,0,1,0).

Обработка столкновений:

(0,0) = 9: две частицы (1,0,0,1) → (0,1,1,0) = 6.

(0,1) = A: две частицы (1,0,1,0) → (0,1,0,1) = 5.

Задача 9. Длинное деление

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	8 мегабайт

Во входном файле задано два целых положительных числа, одно в первой строке, второе — во второй строке. Нужно вычислить частное этих двух чисел (округлённое вниз) и вывести его в выходной файл.
Количество десятичных цифр в каждом входном числе может достигать 1 000. Поэтому требуется реализовать деление двух длинных чисел “столбиком”.

Примеры

input.txt	output.txt
1023405633 84037	12178
1000000000000000000000000000 7	1428571428571428571428571 4

Задача 10. Длинное деление +

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	8 мегабайт

Во входном файле задано два целых положительных числа, одно в первой строке, второе — во второй строке. Нужно вычислить частное этих двух чисел (округлённое вниз) и вывести его в выходной файл.
Количество десятичных цифр в каждом входном числе может достигать 25 000.
Для решения этой задачи необходимо хранить не по одной десятичной цифре в каждом элементе массива, а хотя бы по четыре цифры, в некотором смысле храня длинное число в 10 000-ичной системе исчисления. В процессе деления столбиком нужно на каждом шаге подбирать цифру ответа, и делать это перебором 10 000 возможных вариантов уже неприемлемо.
Определение цифры следует осуществлять бинарным поиском.

Примеры

input.txt	output.txt
1023405633 84037	12178
1000000000000000000000000000 7	1428571428571428571428571 4

Задача 11. Длинное деление ++

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	8 мегабайт

Во входном файле задано два целых положительных числа, одно в первой строке, второе — во второй строке. Нужно вычислить частное этих двух чисел (округлённое вниз) и вывести его в выходной файл.
Обозначим количество десятичных цифр в первом числе через N, а во втором — через M. Тогда верны следующие ограничения:

- $N, M \leq 10^6$
- $M \cdot (N - M) \leq 2.5 \cdot 10^9$

Для решения этой задачи необходимо:
1. Хранить в каждом элементе массива по 9 десятичных цифр (для промежуточных результатов нужны 64-битные целые числа).
2. Находить каждую цифру частного за $O(1)$ попыток против $O(log_2 B)$ попыток для бинарного поиска (здесь $B = 10^9$).
3. Использовать алгоритм с общей асимптотикой $O(M(N - M))$, а не просто любое квадратичное решение.

Примеры

input.txt	output.txt
1023405633 84037	12178
1000000000000000000000000000 7	1428571428571428571428571 4

Задача 12. Поиск подстрок с расстоянием Левенштейна не более 1

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	10 секунд
Ограничение по памяти:	разумное

Нужно реализовать систему поиска подстрок в тексте, где подстрока может отличаться от заданного шаблона не более чем на одну операцию (замена, вставка или удаление символа, т.е. расстояние Левенштейна ≤ 1). Для эффективности рекомендуется использовать Z-функцию. Система должна обрабатывать два типа запросов:

1. 0 — Задать текст: указывается текст, который будет использоваться для последующих поисков.
2. 1 — Найти подстроку: указывается шаблон и выводятся все позиции в тексте, где есть подстрока с расстоянием Левенштейна ≤ 1 от шаблона. Позиции выводятся в порядке возрастания.

Гарантируется, что текст задан перед любым запросом на поиск.
Подсказка: чтобы получить O(n) используйте поиск с двух строн. $l_1 + l_2 + 1 = |P|$ вам как раз говорит о том, что вы с двух сторон подошли к центральному символу.

Формат входного файла

В первой строке записано целое число N — количество запросов ($1 \le N \le 10^6$). В остальных N строках описаны запросы, по одному в строке. Каждый запрос начинается с целого числа t, обозначающего тип запроса:

- Если $t = 0$, то это запрос задания текста. Далее указан сам текст (строка из печатаемых символов ASCII с кодами от 33 до 126 включительно, без пробелов).
- Если $t = 1$, то это запрос поиска. Далее указан шаблон (строка из печатаемых символов ASCII с кодами от 33 до 126 включительно, без пробелов).

Длина текста по запросам $t = 0$ не превышает 10^6.

Длина текста по запросам $t = 1$ не превышает 10^6.

Формат выходного файла

Для каждого запроса типа $t = 1$:
Выведите k пар целых чисел — позиции (начиная с 0) в тексте, где начинаются такие подстроки и расстояние Левенштейна.

Пример

input.txt	output.txt
2 0 abbcabcdexyzab 1 abc	0 1 1 1 4 0 12 1
5 0 abcdexyzab 1 abc 1 abd 0 kotic 1 kot	0 0 8 1 0 1 8 1 0 0

Задача 13. Aho-Corasick

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать систему поиска нескольких шаблонов в тексте. Используйте алгоритм Ахо-Корасика для построения бора (trie) из шаблонов и последующего поиска всех вхождений шаблонов в тексте за один проход. Для оптимизации используйте суффиксные ссылки и ссылки на ближайший терминал для быстрого вывода всех совпадений.

Формат входного файла

В первой строке записано целое число N — количество слов $1 \leq N \leq 10^6$. В остальных N строках описаны слова, по одному в строке. Каждое слово имеет длину $1 \leq word_len \leq 100$. Далее с новой строки следует текст $1 \leq text_len \leq 10^6$. Весь файл состоит из печатаемых символов ASCII с кодами от 33 до 126 включительно, без пробелов).

Формат выходного файла

Выведите для каждого слова его позиции (начиная с 0) в тексте, где оно заканчивается. Позиции выводятся в порядке возрастания.

Пример

input.txt	output.txt
3 Of London the Yes, London is the capital of Great Britain. It's the political, economic, and cultural hub of the United Kingdom, home to landmarks like the Houses of Parliament, Buckingham Palace, and the British Museum. If you have a specific question about London or Great Britain, let me know!	11 18 53 98

Задача 14. RegexToDFA

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Мы знаем, что любое регулярное выражение можно представить в виде детерминированного конечного автомата (DFA).

Ваша задача — построить таблицу переходов детерминированного конечного автомата для данного регулярного выражения. ***(Таблица определяется не однозначно, поэтому сдача происходит лично)***

Формат входного файла

Строка содержит регулярное выражение R, длиной не более **100 символов**, составленное из: строчных латинских букв a-z, операций |, *, +, ?, круглых скобок (), диапазонов символов в квадратных скобках [a-z].

Формат выходного файла

Выведите таблицу как в примере.

Пример

input.txt	output.txt
(a|[a-z])b+?	| a a z ---+---------- 0 | какие-то 1 | состояния